

根号算法选讲

Part I

wYYSZL

2026.8.10

Outline

主要讲分块思想，包括朴素的分块思想、块状数组、操作分块、莫队、根号分治等。
分块似乎也是技巧 > 算法，所以大多数内容都是题目。
假定大家对于入门组的内容已经比较熟练了，所以这部分内容会讲得快一点。
带星的题目是选做题目，但是多数也不会特别难。
欢迎大家挑战它们。

本节课内容

① 基础分块思想

② 块状数组

③ 根号分治

目录

① 基础分块思想

② 块状数组

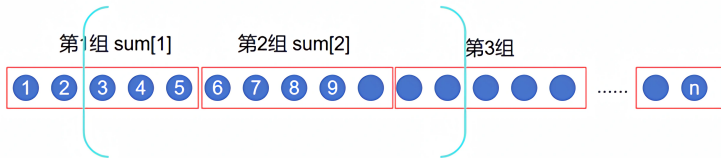
③ 根号分治

基础分块思想

分块是暴力重构和懒标记的平衡。

分块的基本思想是，通过对原数据的适当划分，并在划分后的每一个块上预处理部分信息，从而较一般的暴力算法取得更优的时间复杂度。

当产生例如区间修改的操作时，遇到区间内的整块就打标记，遇到非整块就暴力修改。



基础分块思想

看这样一道例题：

例题

一个长度为 n 的序列 a 。 m 次操作，区间加或区间求和。

$n, m \leq 10^5$

基础分块思想

看这样一道例题：

例题

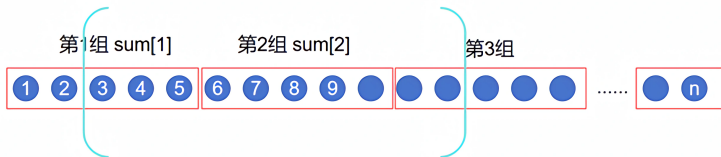
一个长度为 n 的序列 a 。 m 次操作，区间加或区间求和。

$$n, m \leq 10^5$$

当然我们可以用线段树，这里用来引入分块。

考虑我们将序列 a 按照每连续 $\sqrt{n}$ 个元素分一块，剩下的元素组成一块，并记录每个块的区间和 sum_x 。显然，总的块数为 $\frac{n}{\sqrt{n}} = \sqrt{n}$ 个（或者说 $\sqrt{n}$ 级别）。

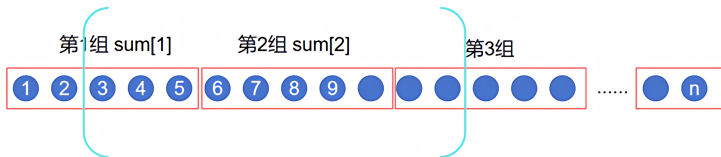
基础分块思想



首先看查询操作：

- 若 l 和 r 在同一个块内，直接暴力求和即可，复杂度为块长度 $O(\sqrt{n})$ 。
- 若 l 和 r 不在同一个块内，则答案由三部分组成： l 所在的不完整块，中间几个完整块，以 r 结尾的不完整块。对于不完整的块，仍然采用上面暴力计算的方法，对于完整块，则直接利用已经求出的 sum_i 求和即可。这种情况下，复杂度为两倍块长 + 总的块个数，于是 $O(\sqrt{n})$ 。

基础分块思想



接下来是修改操作，其实非常类似：

- 若 l 和 r 在同一个块内，直接暴力修改即可，块长为 $\sqrt{n}$ ，因此复杂度为 $O(\sqrt{n})$ 。注意，此时要修改对应块的 sum ，因为查询要用到。
- 若 l 和 r 不在同一个块内，则需要修改的也是三部分。对于不完整的块，仍然是暴力修改每个元素的值并改 sum ，对于完整块，则直接修改 sum_i 即可。这种情况下，复杂度仍然为 $O(\sqrt{n})$ 。

基础分块思想

这样，我们就在 $\mathcal{O}(m\sqrt{n})$ 的时间内解决了这个问题。

现在问题来了，为什么我们要用 $\sqrt{n}$ 这样的块长？

关键在于平衡复杂性以获得最优的总复杂度。我们设块长为 s ，则修改的复杂度为 $\mathcal{O}(2s + \frac{n}{s})$ （两个块长 + 块个数）。我们忽略常数，即 $\mathcal{O}(s + \frac{n}{s})$ 。利用基本不等式，有：

$$s + \frac{n}{s} \geq 2\sqrt{s \times \frac{n}{s}} = 2\sqrt{n}$$

当仅当 $s = \frac{n}{s}$ 时取最值，即 $s = \sqrt{n}$ 。

这种分析技巧应当掌握，有些题目中最佳块长不再是 $\sqrt{n}$ 而需要具体分析。

（如果没学过基本不等式，可以简单理解为积一定时，差越小和越小，或者直接记住两个相等最优也可以）

基础分块思想 - 分块的优势

这道题可以线段树，没有看出分块的优势。那我们为什么要用分块？

分块的主要作用有两个，一是**平衡复杂度**，二是维护一些 $\log$ 数据结构无法维护的信息。

- 对于区间加法，单点查询，一般的思路是使用树状数组维护。此时修改和查询的复杂度均为 $\log n$ 。分块的优势在于它可以让修改和查询当中的任意一个变为 $\mathcal{O}(1)$ ，另一个变为 $\sqrt{n}$ 。当询问或修改的次数非常多时，如莫队二次离线算法中的 $\mathcal{O}(n)$ 次区间修改， $\mathcal{O}(n\sqrt{n})$ 次单点查询，就可以使用分块平衡复杂度，做到非常优秀的 $n\sqrt{n}$ 。
- 当遇到无法快速合并和快速删除的信息时，传统的线段树就失效了。但分块仍然可以做到优秀的复杂度。

相当抽象，我们之后会有例题。

目录

① 基础分块思想

② 块状数组

③ 根号分治

块状数组

很奇怪的名称。似乎没有一个严格定义。

块状数组的含义大概是，在一个块内维护这个块内的信息。通常这个信息量比较大而不是 $\mathcal{O}(1)$ 的。比如维护这个块内的全部某种特征的值。一般信息量只要控制在 $\sqrt{n}$ 之内即可。

我们具体看例题。

例题 A

A - 教主的魔法

给定长度为 n 的整数序列 $a_1, \dots, a_n$, 支持两种操作:

- 区间加: $[l, r]$ 内每个数增加 w ;
- 区间查询: 求 $[l, r]$ 中满足 $a_i \geq c$ 的元素个数。

输出所有查询答案。

$n \leq 10^6$, $q \leq 3000$ 。

例题 A 题解

考虑分块。每个块内维护排好序的块内元素。

区间加时，对于散块（不完整的块），暴力修改并重新排序。如果修改整个块，则直接打一个标记，因为整块修改不改变大小关系，所以不需要重新排序。

查询时，散块暴力看，每个整块在有序副本上做一次 `lower_bound`。

听起来非常暴力，但是实际复杂度比暴力好，为 $\mathcal{O}(q\sqrt{n}\log n)$ 。

实现时，注意不能给 `a` 数组排序，而是应该复制出去用另一个数组排序。

`lower_bound` 时不要忘记整块加的 `tag`。

线段树与分块

为什么不能直接用线段树？

原因在于， $\geq c$ 这样的信息是没有办法快速合并的。也就是说，如果知道左右儿子的信息，我们没法快速知道父亲的信息。同时，我们也无法快速地直接从修改中得到本节点的答案。

而分块不存在这样的问题。分块可以接受暴力的“合并”，对于这道题就是允许暴力地重新排序。这使得分块的适用范围远多于线段树等数据结构。

此外

分块可以看作只有两层的多叉线段树，因而线段树可以实现的操作都可以用分块以较劣的复杂度实现。有时由于分块的小常数，时间上分块甚至优于线段树。

例题 B

B - 作诗

给定 n 个不大于 c 的正整数 $a_1 \dots a_n$ 和 m 组询问, 每次问 $[l, r]$ 中有多少个数出现正偶数次。

强制在线, $1 \leq n, c, m \leq 10^5$

例题 B 题解

我们对整块的问题预处理好答案，设 $f_{i,j}$ 为询问区间从第 i 个块开头到第 j 个块末尾时的答案。

求 $f_{i,j}$ 可以从第 i 块开头，一次性求出所有 j 的 f 值，一次需要 $\mathcal{O}(n)$ 的时间，总计需要 $\mathcal{O}(n\sqrt{n})$ 。

有了这个答案之后，我们只需求散块对答案的影响。我们对每个不同的值开一个 vector，如果一个值在散块中出现了，那么它会对答案产生影响。这时查询它在 $[l, r]$ 区间内的出现次数以及在整块内的出现次数：

- 如果在整块内出现偶数次但是在区间内出现奇数次，则答案 -1 。
- 如果在整块内出现奇数次但是在区间内出现偶数次，则答案 $+1$ 。
- 此外不变。

这样，每次查出现次数需要两次二分，总的复杂度 $\mathcal{O}(n\sqrt{n}\log n)$ 。但这不够。

例题 B 题解

我们还可以空间换时间。我们对于每个数值，维护其在 $i \sim j$ 块内的出现次数。这样复杂度爆炸。可以用前缀和。

具体地，用 $pre_{x,y}$ 表示数 x 在前 y 个块内的出现次数。这样，每次询问时先处理出散块各个数的出现次数，同时结合这个 pre 数组就能 $\mathcal{O}(1)$ 求出散块内一个数在 $[l, r]$ 的出现次数。

总的时间、空间复杂度 $\mathcal{O}(n\sqrt{n})$ 。

Hint

通常来讲，关于“出现次数”等东西难以实现 Polylog，要想到根号算法。

例题 C

C - 弹飞绵阳

给定长度为 n 的数组 $k_0, \dots, k_{n-1}$ ($k_i \geq 1$)，定义从位置 i 出发，每次跳到 $i + k_i$ ，若越界则停止。支持两种操作：

1. 查询：给定 j ，输出从 j 出发到停止所需的跳跃次数；
2. 修改：给定 j 和 r ，将 k_j 改为 r 。

数据规模： $n \leq 2 \times 10^5$ ， $m \leq 10^5$ 。

例题 C 题解

不难转化为加边/删边动态求深度，考虑使用 LCT 解决。

分块的精髓在于平衡查询与修改，直接的想法是每次 $\mathcal{O}(1)$ 修改然后暴力查询，给修改上复杂度是每次修改从后向前求每个点停止需要的跳跃次数， $\mathcal{O}(n)$ 修改然后 $\mathcal{O}(1)$ 查询。

稍微平衡就是，我们对序列分块，每个点维护跳出本块的弹跳次数和下一步的位置。每次修改时从后向前暴力计算本块内的点跳出本块的次数与位置，询问时每次跳跃都至少跳出一个块。两个复杂度都是 $\mathcal{O}(\sqrt{n})$ 。

整个过程听起来类似倍增。确实是这样的。而且这个做法和倍增的关系，就类似线段树和分块的关系。

目录

① 基础分块思想

② 块状数组

③ 根号分治

根号分治

根号分治本质上是一种 按规模大小分类讨论的思想而非分治算法。这里的“分治”表示分类而不是通常的“分而治之”的含义。对于规模为 x 的问题，如果我们能在 $\mathcal{O}(x)$ 和 $\mathcal{O}(\frac{n}{x})$ 的时间内解决，可以考虑根号分治：

- $x \leq \sqrt{n}$ 时使用 $\mathcal{O}(x)$ 的算法
- $x > \sqrt{n}$ 时使用 $\mathcal{O}(\frac{n}{x})$ 的算法

类似前面的，这相当于寻找 x 和 $\frac{n}{x}$ 的较小值的最大值：显然，当 $x = \sqrt{n}$ 时， $\min(x, \frac{n}{x})$ 取到最大值 $\sqrt{n}$ 。

说起来有点抽象，我们来看例题。

例题 D

D - Time to Raid Cowavans

给定长度为 n 的数组 $w_1, \dots, w_n$, 以及 p 个询问, 每个询问给出参数 (a, b) 。
定义一次询问的答案为:

$$\sum_{t \geq 0, a+tb \leq n} w_{a+tb}$$

依次输出每个询问的答案。

$n, p \leq 3 \times 10^5$ 。

例题 D 题解

当步长 b 比较大的时候，由于跳不了几步，所以我们可以直接暴力解决，时间复杂度是 $\mathcal{O}(\frac{n}{b})$ 。

当 b 比较小时，我们从后向前，预处理出所有 a 对应的答案（类似后缀和的做法）。设比较小定义为 $b \leq s$ ，则这个的总复杂度为 $\mathcal{O}(sn)$ 。

不妨认为 n, p 同阶，则取 $s = \sqrt{n}$ ，有总的复杂度为 $\mathcal{O}(n\sqrt{n})$ 。

非常不牛的是本题卡空间常数，可以离线下来，按照 b 从小到大的顺序处理询问。 b 比较小的时候只需要存储当前 b 的答案即可，空间复杂度降到 $\mathcal{O}(n)$ 。

例题 E

E - Till I Collapse

给定长度为 n 的数组 $a_1, \dots, a_n$ ($1 \leq a_i \leq n$)。对于每个 $k = 1, 2, \dots, n$, 定义 $f(k)$ 为将序列划分为若干个连续段, 使得每段内不同数值的个数不超过 k 所需的最少段数。输出 $f(1), f(2), \dots, f(n)$ 。

$n \leq 10^5$ 。

例题 E 题解

首先考虑暴力。对于一个左端点，让右端点向右。然后这个右端点的下一个位置作为左端点。这个贪心显然是正确的。

之后依然考虑根号分治。

- 对于 $k \leq \sqrt{n}$ ，直接暴力解决。
- 对于 $k > \sqrt{n}$ ，注意到答案不超过 $\frac{n}{k}$ ，所以这时答案不超过 $\sqrt{n}$ 。先求出来 $k = \sqrt{n}$ 的答案，然后从左到右，依次暴力让每个右端点不停向右走直到走不动。

考虑第二类的复杂度。由于右端点只有 $\sqrt{n}$ 个，所以每次考虑 $\mathcal{O}(\sqrt{n})$ 个点，并且总共不会移动超过 $n\sqrt{n}$ 。

总的时间复杂度 $\mathcal{O}(n\sqrt{n})$ 。

例题 E 另解

本题还有另一种复杂度稍劣但是值得一说的根号分治做法。

其他内容都和前面说到的一样，唯一不同的是 $k > \sqrt{n}$ 的做法。这个做法观察到了答案的单调性，随 k 增大，答案单调不减。

因而在求出一个 k 的答案后，二分这个答案的边界。check 的时候暴力算，复杂度 $\mathcal{O}(n)$ 。总的复杂度根号 $\log$ 。

但这个复杂度可以通过调整分界点优化。设分界点不是 $\sqrt{n}$ 而是 s ，则第一种复杂度 $\mathcal{O}(sn)$ ，第二种 $\mathcal{O}(n \times \frac{n}{s} \log n)$ ，二者乘积一定，当：

$$sn = \frac{n^2}{s} \log n$$

即 $s = \sqrt{n \log n}$ 时，总复杂度最优，为 $\mathcal{O}(n\sqrt{n \log n})$ 。这种技巧比较常见。

例题 E 另解

本题还有双 $\log$ 的做法。有一些值得一提的小 Trick。

一个经典技巧：统计一个区间颜色数，我们在某个数第一次出现在这个区间时统计。

维护 lst_i 表示上一个和 a_i 相同的颜色的出现位置，则答案就是 $lst_i < l$ 的个数。

我们刚才注意到答案不会超过 $\frac{n}{k}$ ，而 $\sum \frac{n}{k}$ 是调和级数为 $n \log n$ 级别，也就是总的区间数不超过这个级别。

考虑优化暴力，每次二分右端点，check 时就是查 $[l, r]$ 中 $lst_i < l$ 的个数，一个二维数点问题，可以可持久化线段树。

不会没关系，也没有必要。我们可以对所有 k 同时处理。具体地，从小到大枚举 l ，设此时 $[1, l-1]$ 已经划分好，且 $l-1$ 恰是一个段的右端点，现在等待找左端点为 l 的右端点的 k 。由于知道了 l ，所以 $lst_i < l$ 对于一个 i 就是一个 1/0 的事情。这个东西树状数组即可。

总的时间复杂度 $\mathcal{O}(n \log^2 n)$ 。

例题 F

F - Regions

给定一棵 n 个节点的有根树（根为 1），每个节点有一个颜色 $c_u \in [1, R]$ 。

对于每个询问 (r_1, r_2) ，求满足以下条件的节点对 (u, v) 的数量：

- u 是 v 的祖先；
- $c_u = r_1, c_v = r_2$ 。

$n, Q \leq 2 \times 10^5, R \leq 2.5 \times 10^4$ 。4s，强制在线。

例题 F 题解

这种集合点对或者和“出现次数”有关的也是非常强的根号算法信号。

考虑对颜色 i 的出现次数根号分治，如果出现次数大于 C 则称为大集合，否则为小集合。分类：

- 如果 r_1 为大集合，预处理其他所有可能的 r_2 的答案。我们希望复杂度依赖 r_2 的大小，可以预处理每一个点有多少个 r_1 祖先。具体的，答案为 $\sum_{c_i=r_2} t_i$ ， t_i 表示 i 的祖先中有多少颜色为 r_1 。一个颜色复杂度 $\mathcal{O}(n)$ 。
- 如果 r_2 为大集合，对称。
- 如果两个都是小集合。可以暴力做下去。对于每一个 r_1 的点要求子树区间内有多少 r_2 颜色的点，用 DFS 序转化一下就是区间问题。对于每个 r_2 单点 $+1$ ，对于每个 r_1 区间求和。树状数组即可。

例题 F 题解

类似上一题，我们可以优化 C 获得 $\mathcal{O}(n\sqrt{n\log n})$ 的复杂度。

但我们可以优化，注意到两个小集合时的单点修改区间查询是全部修改完然后查询，实际上是离线的。所以直接离散化后前缀和就行。

但是离散化不能 `sort` 排序。我们需将每个颜色的点排序好，然后类似归并排序合并即可。总的时间复杂度 $\mathcal{O}(n\sqrt{n})$ 。

小节

最后实际看来，根号分治通常是在 **预处理**和 **询问**的复杂度之间寻找平衡，和一开始的定义有一定出入。

根号分治还有另一类应用。利用根号分治思想，我们不难证明这一点：

数论分块

对于固定的 n ， $\lfloor \frac{n}{i} \rfloor$ 只有 $\mathcal{O}(\sqrt{n})$ 个取值。

由是，我们可以快速计算形如：

$$\sum_{i=1}^n f(i)g\left(\left\lfloor \frac{n}{i} \right\rfloor\right)$$

的式子。但这个东西超出了我们要讨论的内容，故掠过。